Génie électrique et informatique industrielle

université de BORDEAUX

Travaux pratiques Automatique

Semestre <4>

<TP1 et TP2>

Date 26/02/2026

Thiaw Kouthieye
Coppi Joris

Groupe : B1b

COMPTE-RENDU : RÉGULATION D'UN SYSTÈME ÉLECTROMÉCANIQUE

1. Présentation du système

Le système étudié est un dispositif électromécanique. Nous l'avons caractérisé lors d'une précédente analyse. Une carte avec un microcontrôleur STM32 gère la commande et les calculs. L'objectif est d'asservir la vitesse du moteur pour lui faire suivre une consigne.

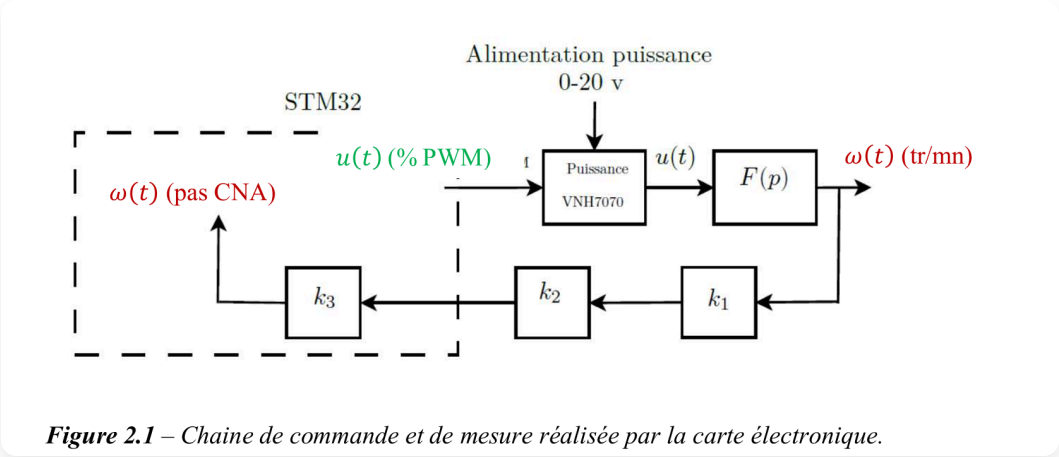


Figure 2.1 *– Chaine de commande et de mesure réalisée par la carte électronique.*

Figure 1 - Schéma de principe du bloc moteur piloté par STM32.

Les mesures du système donnent un modèle mathématique direct. Le moteur se comporte comme un système du premier ordre. Le gain statique vaut $K = 32$. La constante de temps est évaluée à $\tau = 0.02$ seconde. Nous écrivons la fonction de transfert en boucle ouverte $H(p)$:

$$H(p) = \frac{32}{1 + 0.02 \cdot p}$$

Le moteur agit mécaniquement comme un filtre passe-bas. L'inertie du rotor limite ses performances de vitesse. Le moteur ne peut pas suivre de variations trop rapides.

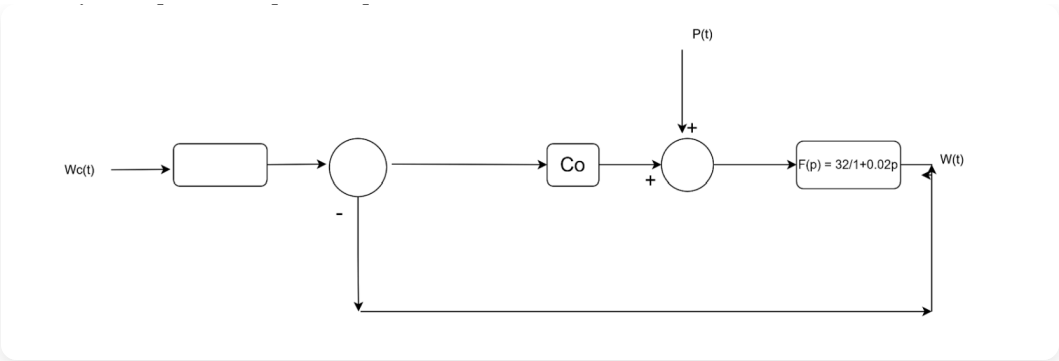


Figure 2 - Architecture de la boucle fermée avec correcteur proportionnel.

2. Commande proportionnelle

Nous introduisons une boucle de retour unitaire et un correcteur proportionnel. Le correcteur possède un gain multiplicateur C_0. Le système calcule l'écart entre la consigne d'entrée et la mesure du capteur. Cette erreur est multipliée par C_0 pour générer la commande du moteur.

Nous utilisons le Théorème de Mason pour étudier le comportement global de la boucle d'asservissement. Ce théorème relie la chaîne d'action à la boucle de retour. Il donne directement la fonction de transfert en boucle fermée modélisée :

$$FTBF(p) = \frac{C_0 \cdot H(p)}{1 + C_0 \cdot H(p)}$$

Le gain C_0 définit totalement la rapidité et la dynamique du système. Nous observons sa répercussion avec une étude de gain harmonique sur Matlab.

TP1S3.m

/MATLAB Drive/TP1S3.m

```
num =32;
den =[0.02, 1];
Hp = tf(num, den);
figure;
bode(Hp);
```

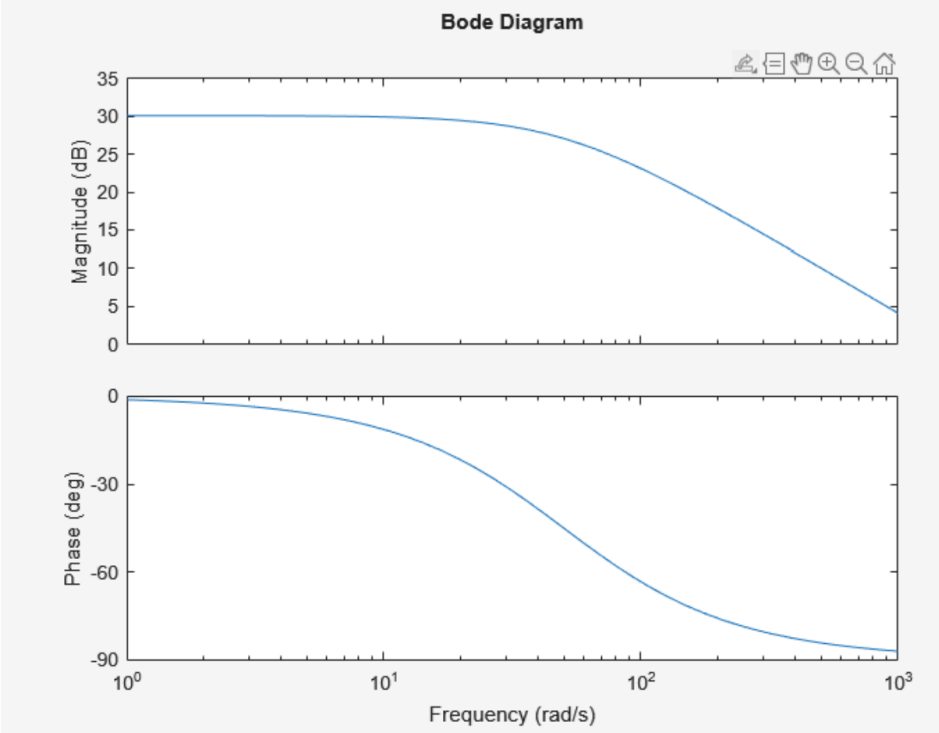


Figure 3 - Script de tracé et modélisation fréquentielle de l'installation sous Bode.

2.1 Limites physiques et saturation

Le cahier des charges impose une réponse très rapide. Le temps de montée exigé est de $t_m = 1\,\mu$s. Nous calculons la bande passante nécessaire pour obtenir cette vitesse extrême :

$$\omega_u \approx \frac{3}{10^{-6}} = 3 \cdot 10^6\,\text{rad/s}$$

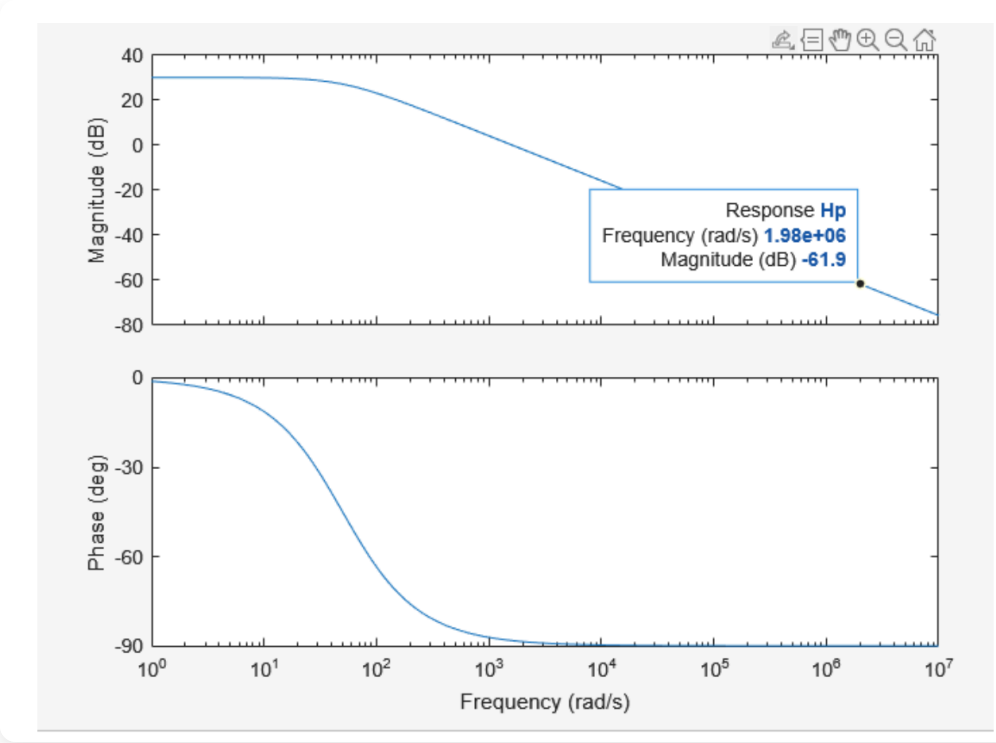


Figure 4 - Évaluation de la réponse à très haute fréquence.

Pour atteindre cette pulsation, le gain de la boucle ouverte doit valoir 1 à cette fréquence précise. Nous exploitons l'asymptote et posons l'égalité de norme complexe :

$$\left| C_0 \cdot \frac{32}{\sqrt{1 + (0.02 \cdot \omega_u)^2}} \right| = 1$$

Nous résolvons cette équation et trouvons le paramètre $C_0 \approx 1875$. Nous simulons ensuite l'application d'un échelon de consigne de 2500 pas. La loi de commande évalue la tension d'amorçage à l'instant initial :

$$u(0^+) = C_0 \cdot \varepsilon(0^+) = 1875 \times 2500 = 4\,687\,500\,\%$$

Cette valeur de commande est physiquement absurde. L'alimentation de puissance (hacheur) de la carte STM32 limite fermement le rapport cyclique à 100%. Le système sature donc instantanément. Le temps d'une microseconde est impossible à atteindre en réalité. Nous décidons donc de réduire massivement le gain.

2.2 Erreur statique

Nous choisissons un gain de sécurité fixé à $C_0 = 0.2$. Le hacheur reste dans sa zone linéaire sans saturer. La précision devient alors le nouveau problème majeur du système. Nous utilisons le

Théorème de la Valeur Finale pour calculer l'erreur de régime permanent. Nous simulons un échelon de consigne standard de 2000 pas et écrivons la limite :

$$\varepsilon_{stat} = \lim_{p \to 0} \left(p \cdot \frac{1}{1 + C_0 \cdot H(p)} \cdot \frac{2000}{p} \right)$$

Nous remplaçons la fonction de transfert par ses valeurs nominales. La résolution donne directement l'écart statique terminal de la maquette :

$$\varepsilon_{stat} = \frac{2000}{1 + 0.2 \times 32} = \frac{2000}{7.4} \approx 270.27 \, \text{pas CAN}$$

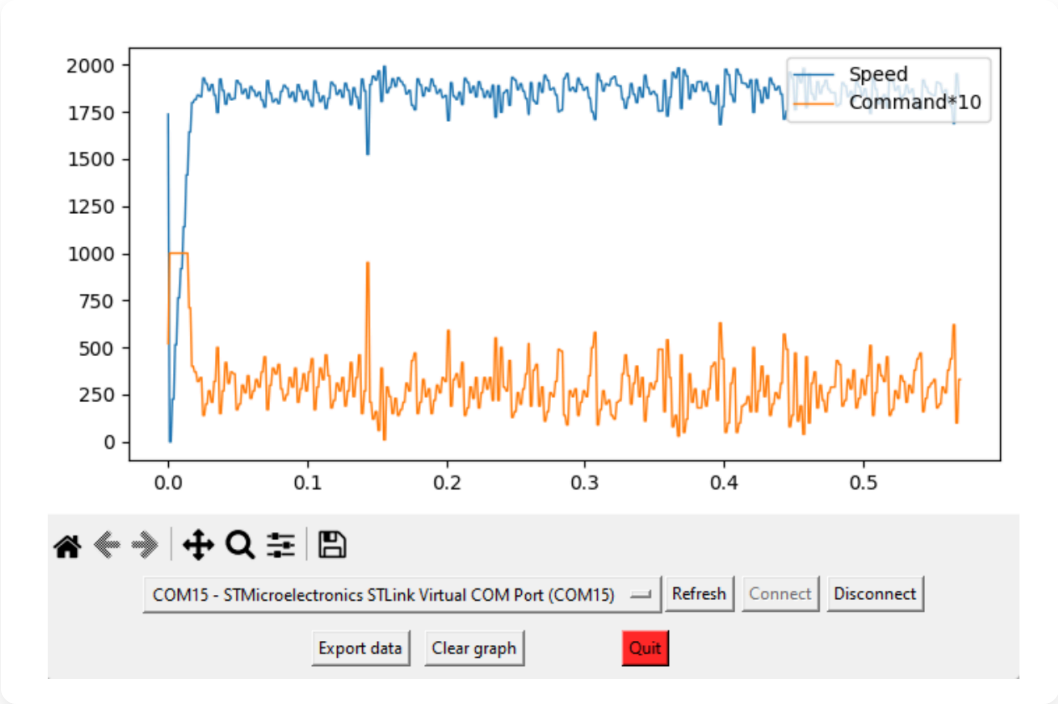


Figure 5 - Trace expérimentale signalant l'écart visuel permanent.

L'expérience sur le banc confirme cette déficience mathématique. La valeur de la vitesse mesurée n'atteint jamais la ligne de consigne. Un correcteur simplement proportionnel génère une erreur statique continue.

3. Commande proportionnelle et intégrale (PI)

Nous insérons une action purement intégratrice au correcteur pour pallier ce défaut. Le bloc d'intégration emmagasine l'erreur résiduelle au fil du temps. Il injecte le retard sur la commande pour réagir en continu. L'équation canonique de notre correcteur PI devient :

$$C_i(p) = C_{0i} \cdot \frac{1 + T_i \cdot p}{T_i \cdot p}$$

TP1S3.m | TP1_S4_Correcteur_PI.m

/MATLAB Drive/TP1_S4_Correcteur_PI.m

```
1
2   numH = 32;
3   denH = [0.02 1];
4
5   wu = 90;
6   Mp = 80;
7
8   [g,p] = bode(numH, denH, wu);
9
10  g = squeeze(g);
11  p = squeeze(p);
12
13  Ti = 1/wu*tan(pi/180*(90-180+Mp-p));
14  Ki = 1/(g*sqrt(1+(Ti*wu)^2)/(Ti*wu));
15  numC = Ki*[Ti 1];
16  denC = [Ti 0];
17
18  H = tf(numH, denH);
19  C = tf(numC, denC);
20  BO = C * H;
21  nichols(BO);
22  ngrid;
```

Figure 6 - Exécution de la routine Matlab pour calibrer les paramètres du correcteur PI.

Le pilotage au moyen du microcontrôleur inflige un déphasage dégradant la stabilité. Nous paramétrons la réponse en imposant une fréquence de coupure à $90\,\mathrm{rad/s}$ et une marge de robustesse de 80°. La vérification du paramétrage s'effectue sur le tracé complexe de Nichols.

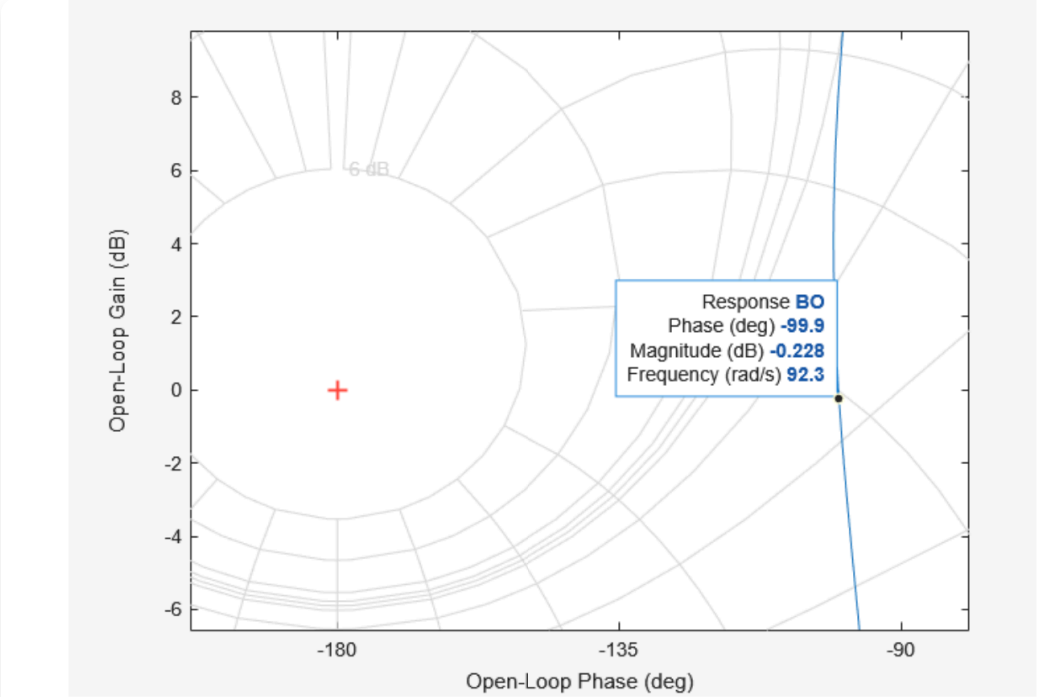


Figure 7 - Validation du respect géométrique de la phase et de la stabilité de la boucle.

Nous justifions ensuite l'effacement de l'erreur par le Théorème de la Valeur Finale. L'intégrateur superpose mathématiquement un pôle à l'origine caractérisé par le terme $1/p$. Le calcul s'écrit de la manière suivante :

$$\varepsilon_{stat} = \lim_{p \to 0} \left(p \cdot \frac{1}{1 + C_i(p) \cdot H(p)} \cdot \frac{2000}{p} \right)$$

Sous la limite au régime établi, la fraction du correcteur impose un démultiplicateur infini sur la dénomination de la boucle de transfert. La division tend fermement vers la nullité totale :

$$\varepsilon_{stat} = \frac{2000}{1 + \infty} = 0$$

Ce calcul démontre la force de l'intégration pure. L'erreur statique disparaît totalement de la mécanique asservie.

4. Implantation numérique

La carte microcontrôleur requiert un langage informatique séquentiel. L'équation de base du correcteur est une donnée continue et différentielle :

$$T_i \cdot \frac{du(t)}{dt} = C_{0i} \cdot \varepsilon(t) + C_{0i} \cdot T_i \cdot \frac{d\varepsilon(t)}{dt}$$

Il est impératif d'utiliser la discrétisation numérique temporelle de la dérivée pour convertir le modèle. L'algorithme remplace la primitive d'évolution par un simple écart soustractif évalué sur un pas fixe T_e . La factorisation informatique isole la commande de l'instant présent $u(k)$. Nous aboutissons à la loi de récurrence formelle :

$$u(k) = u(k-1) + C_{0i}\left(1 + \frac{T_e}{T_i}\right)\varepsilon(k) - C_{0i} \cdot \varepsilon(k-1)$$

```
334     // Régulateur PI
335     err_ = err;   //On sauvegarde l'erreur à l'itération k-1
336     err = r - meas; //On calcule l'erreur à l'itération k-1
337     commande = 0.0536*err - 0.050*err_ + commande;
338     u = (int) commande;
```

Figure 8 - Transposition matérielle par langage C exécutant séquentiellement la récurrence PI.

Nous codons cet algorithme itératif directement dans l'interruption périodique de programmation STM32. Le programme actualise et module la commande PWM en permanence. Le relevé expérimental sur cible prouve le triomphe de l'implantation.

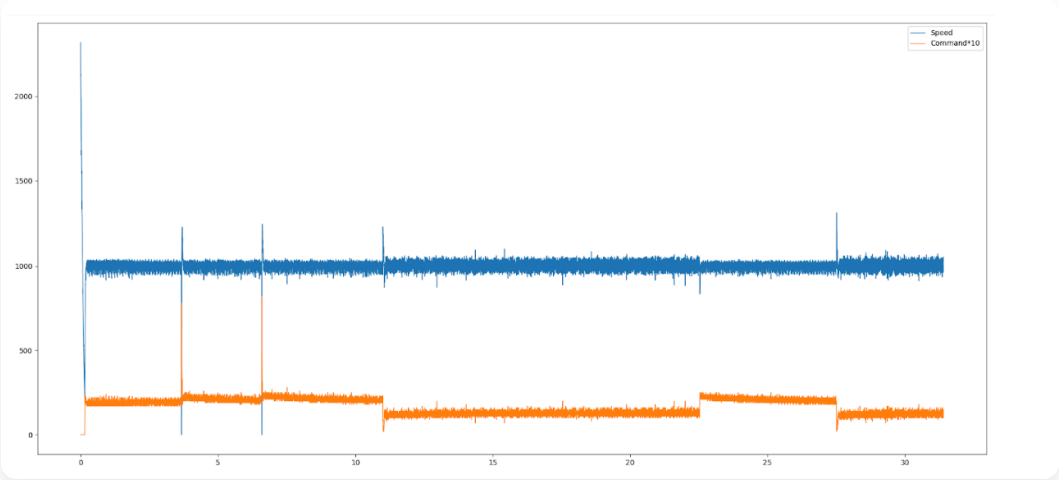


Figure 9 - Affichage du maintien absolu de la consigne face au rejet de perturbation.

La vitesse du moteur est strictement calée sur la courbe de la commande. Au freinage imposé brutal de l'arbre mécanique, la régulation maintient fermement l'objectif rotatif et compense instantanément la distorsion. Un dernier bilan est certifié via le logiciel croisé Simulink.

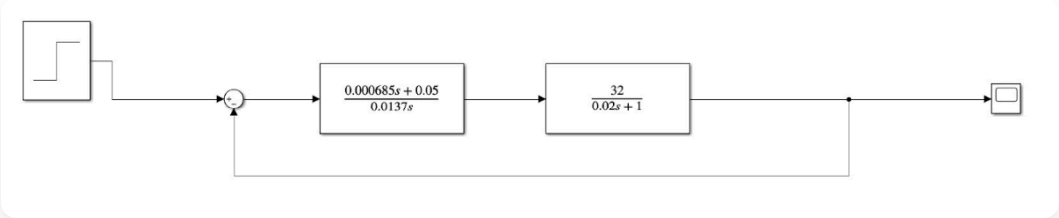


Figure 10 - Organigramme logique Simulink calculant l'asservissement sans friction matérielle.

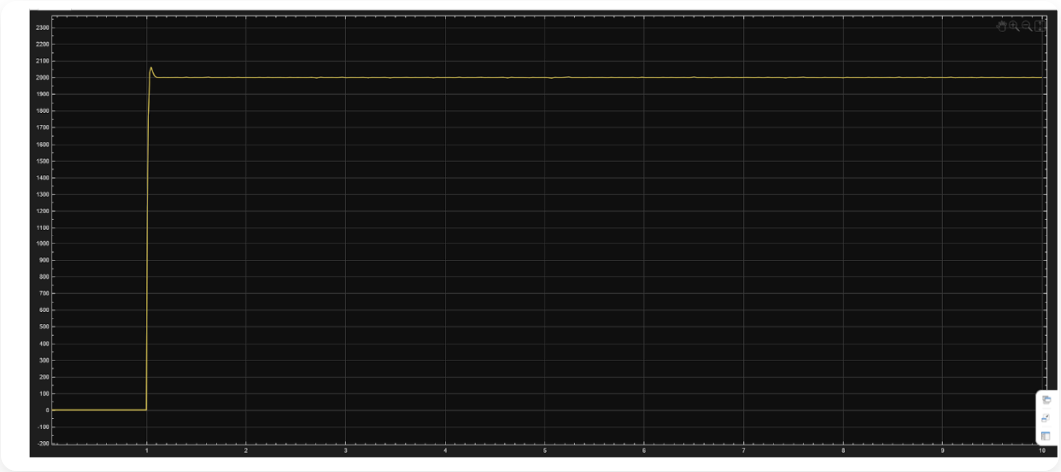


Figure 11 - Tracé d'oscilloscope en simulation évinçant toute existence d'erreur en régime final.

5. Conclusion

Cette étude démontre les défauts capacitaires naturels d'un système électromécanique physique. Une commande strictement proportionnelle crée une saturation du hacheur en mode transitoire puis induit une erreur d'exactitude statique en mode permanent. Le couplage avec une action intégratrice absorbe et annule incontestablement ces erreurs théoriques. La discrétisation numérique temporelle de cette équation offre finalement un moyen informatique très abordable et hautement fiable pour réguler sans faille le moteur et rejeter formellement toute charge perturbatrice externe.

TP2 - Régulation D’un système thermique

Introduction

Le but de ce TP est de simuler et mesurer le comportement de différents correcteurs proportionnels et de correcteurs PI afin de les comparer. Tout en réalisant l’analyse théorique.

1.2 - Commande Proportionnelle

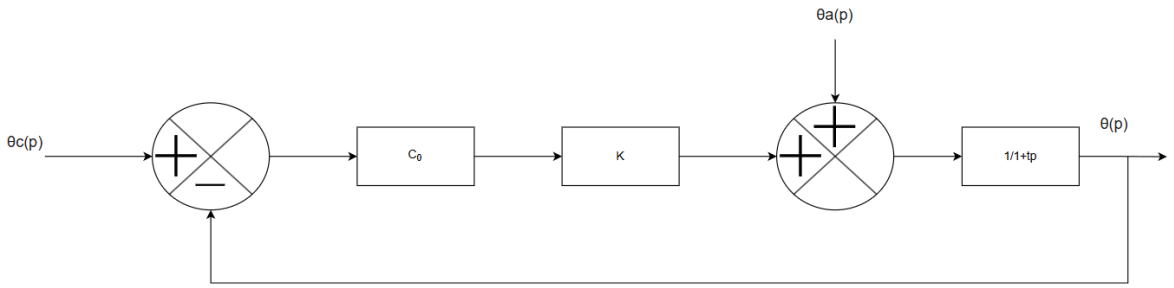


Figure 12 : Schéma bloc en boucle fermée du système thermique

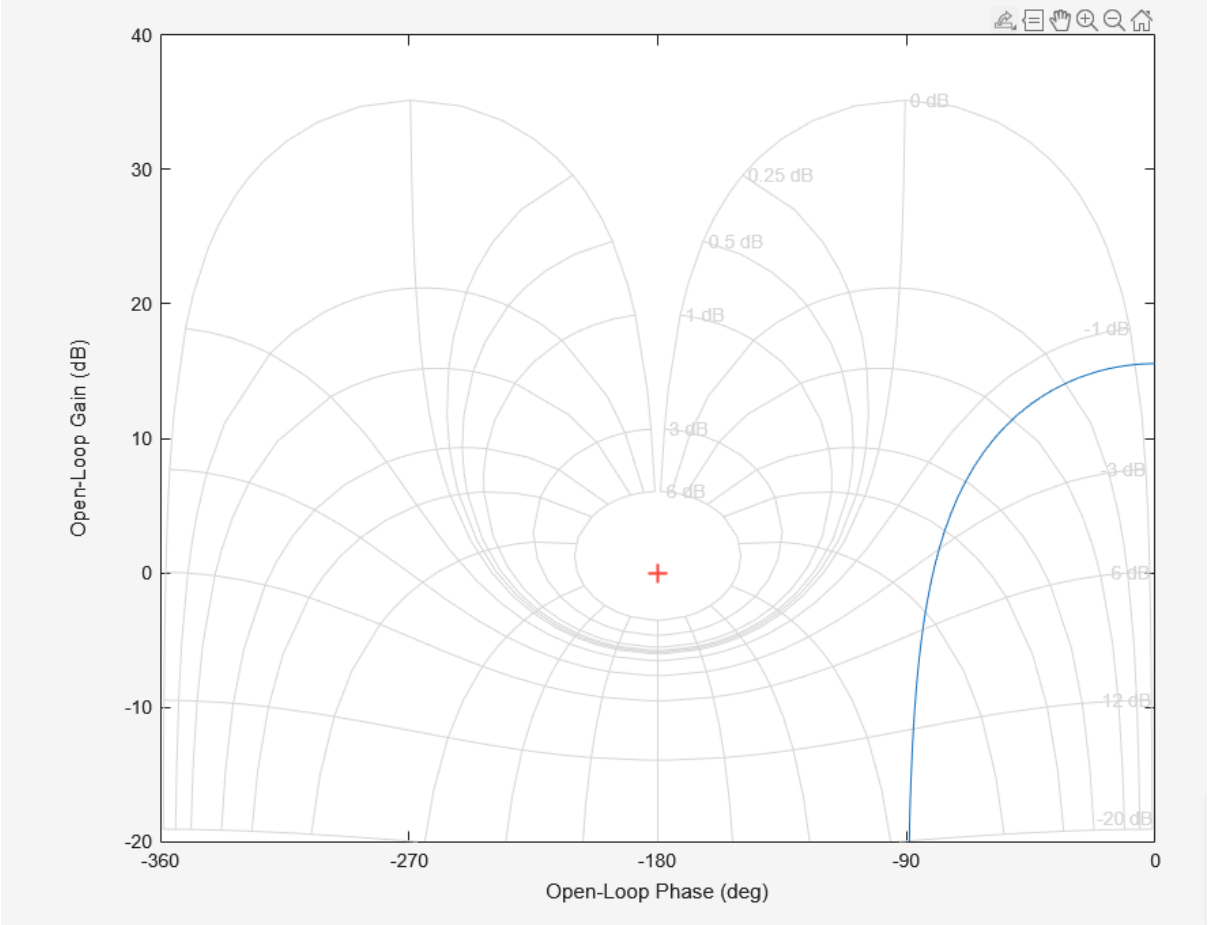


Figure 13 : Diagramme de Black-Nichols du système en boucle ouverte

D'après le diagramme ci-dessus, nous pouvons lire que la marge de phase est de 60°.

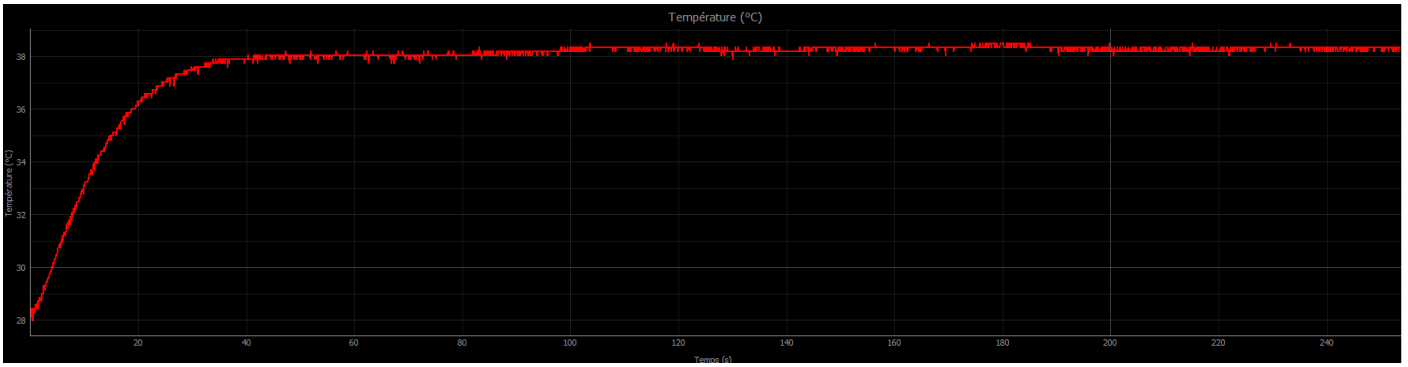


Figure 14 : Graphique de la température en fonction du temps

Nous mesurons, d'après le graphe ci-dessus, le temps de réponse à 5% ainsi que l'erreur statique :

- Temps de réponse à 5% :
  La température initiale est 20°C, la température finale est 38,3°C, la différence est donc de 18,3°C.
  Nous devons donc calculer 5% de 18,3°C soit 0,9°C.
  Il faut maintenant mesurer le temps que va mettre le système pour atteindre Tfin-0,9 soit : 37,4°C,
  Le système met environ 28s pour atteindre 37,4°C
- Erreur statique :
  Nous mesurons la valeur finale atteinte par le système soit 38,3°C. Nous faisons ensuite la différence entre la valeur finale théorique, 40°C, et la valeur finale réelle, 38,3°C.
  L'erreur statique vaut donc : 1,7°C

  Nous calculons maintenant analytiquement l'erreur statique en utilisant le théorème de la valeur finale :

$$lim\Theta(t)_{t\to\infty} = limp\Theta(p)_{p\to 0} = p\left(\frac{\frac{C_0 k}{1+\tau p}}{1+\frac{C_0 k}{1+\tau p}}\Theta_c(p) + \frac{\frac{1}{1+\tau p}}{1+\frac{C_0 k}{\tau p}}\Theta_a(p)\right)$$

  La commande $\theta c(p)$ est un échelons à 40°C, $\theta a(p)$ est la température de la salle (22°C) qui peut être représentée par un échelon.
  On a alors :
  $\theta c(p) = \frac{40}{p}$
  $\theta a(p) = \frac{22}{p}$
  l'équation devient alors :

$$limp\theta(p)_{p\to 0} = p\left(\frac{\frac{C_0k}{1+\tau p}}{1+\frac{C_0k}{1+\tau p}}\frac{40}{p} + \frac{\frac{1}{1+\tau p}}{1+\frac{C_0k}{1+\tau p}}\frac{22}{p}\right)$$

En simplifiant l'équation, on obtient :

$$limp\theta(p)_{p\to 0} = \frac{40C_0k}{1+\tau p+C_0k} + \frac{22}{1+\tau p+C_0k}$$

Or p tend vers 0 donc on obtient :

$$\frac{40C_0k+22}{1+C_0k}$$

En remplaçant par les valeurs numériques, on obtient :

$$\frac{40\times 10\times 0,6+22}{1+10\times 0,6} = 37,43$$

D'après le théorème de la valeur finale, nous devions obtenir une température finale de 37,43°C, nous avons pu mesurer sur les graphes une température de 38,3°C. Ces valeurs comportent une petite différence probablement due aux erreurs de mesure de la température de la salle et de la température du système. Dans les deux cas, nous constatons une erreur statique qui est le principal défaut d'une commande proportionnelle. En revanche, une commande proportionnelle a l'avantage d'être plus simple à mettre en œuvre.

Ensuite, nous réalisons le schéma suivant avec le logiciel simulink :

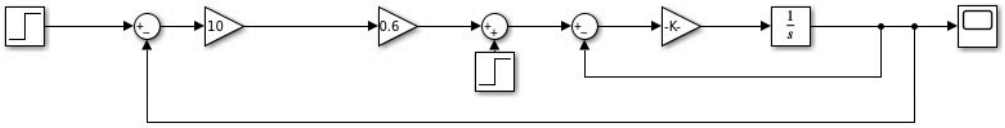


Figure 15 : Schéma bloc pour la simulation simulink

Nous traçons le graphe correspondant à cette fonction de transfert :

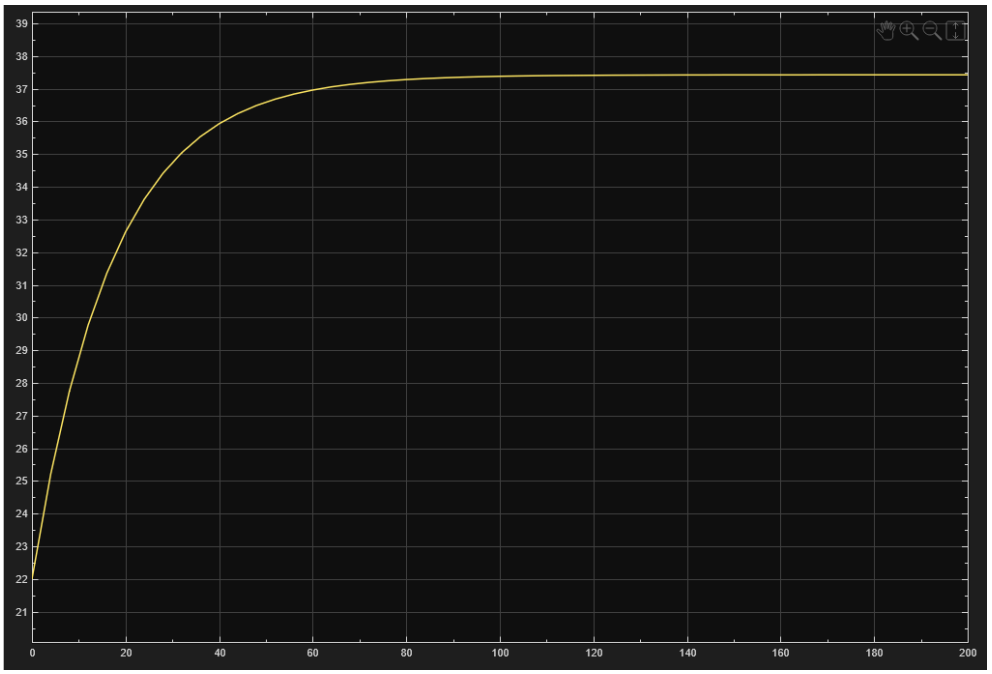


Figure 16 : Graphique de la simulation simulink

On observe d'après le graphe de la figure 16 que la valeur finale est bien 37,4°C conformément à ce que nous avons calculé précédemment.

Nous implémentons le programme dans l'Arduino avec les valeurs correspondant au correcteur proportionnel, nous attendons que le régime permanent soit établi puis nous perturbons le système avec un ventilateur et obtenons le graphe suivant :

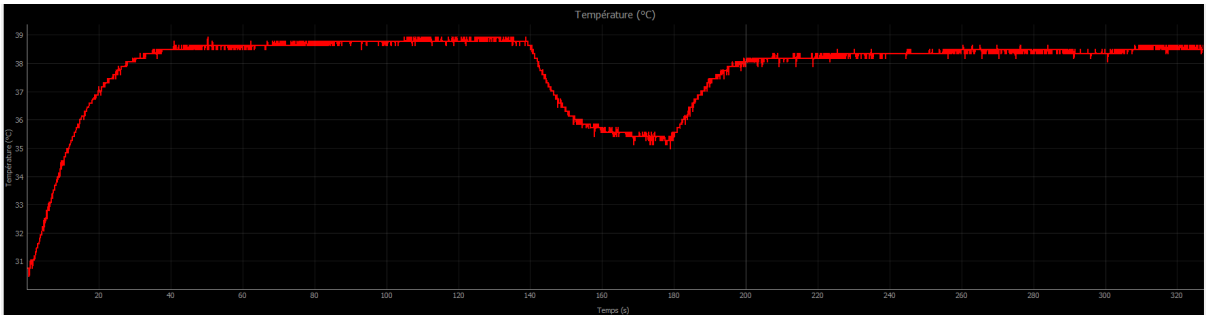


Figure 17 : Graphique de l'essai avec le correcteur proportionnel

Nous observons que la perturbation est en grande partie compensée en environ une minute. Néanmoins, nous observons sur ce graphe que nous n'atteignons pas le régime permanent lorsque le système est perturbé. Nous aurions dû attendre plus longtemps avant d'atteindre le régime permanent. Nous remarquons donc que le système a une difficulté à rétablir le régime permanent lorsqu'il est perturbé.

1.4 - Commande Proportionnelle et Intégrale

A l'aide du logiciel Matlab, nous calculons trois correcteurs proportionnel :

Le premier a une marge de phase de 60° et une pulsation au gain unité en boucle ouverte w_u=0,05 rad/s et nous obtenons les valeurs suivantes :

- C0 = 7,8269
- Ti = 24,2945

Nous obtenons les graphes de phase et de gain suivant :

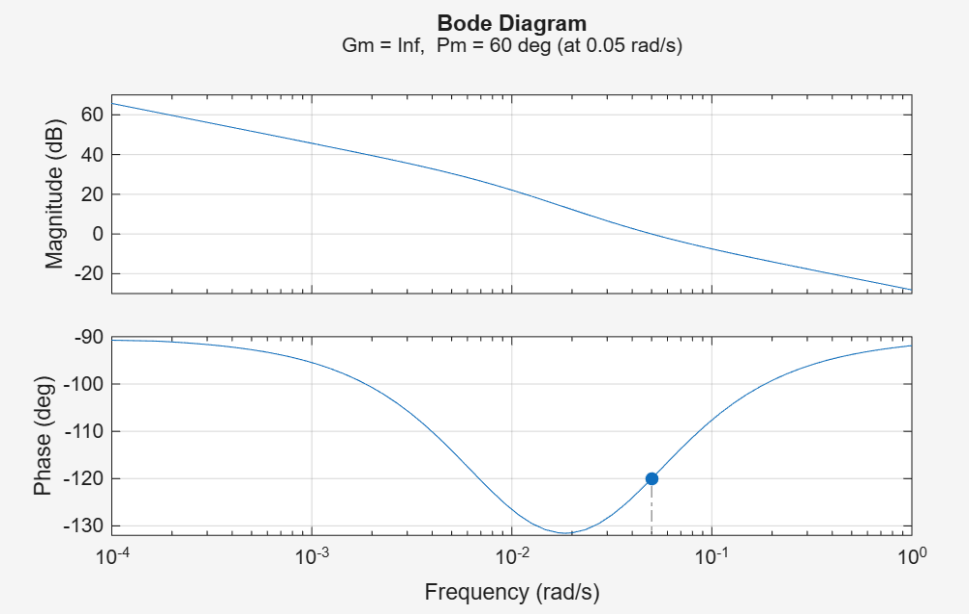


Figure 18 : Simulation Matlab du premier correcteur proportionnel intégral

En effectuant un zoom sur le graphe de la figure 18,, nous voyons que la pulsation au gain unité est bien de 0,05 rad/s et que la marge de phase est effectivement de 60° :

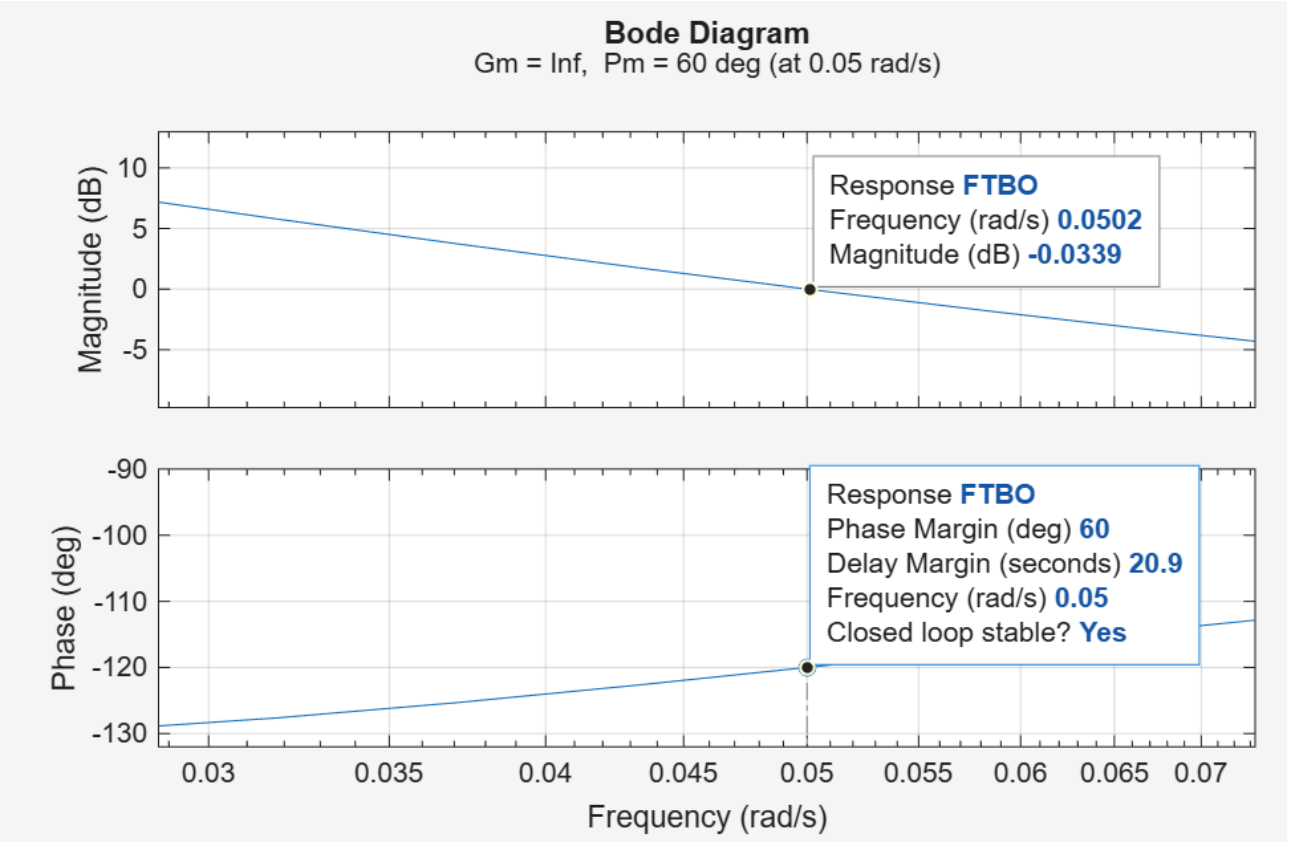


Figure 19 : Simulation Matlab du premier correcteur proportionnel intégral (vue détaillée)

Après avoir simulé, nous implantons le correcteur PI dans le système et obtenons le graphique suivant :

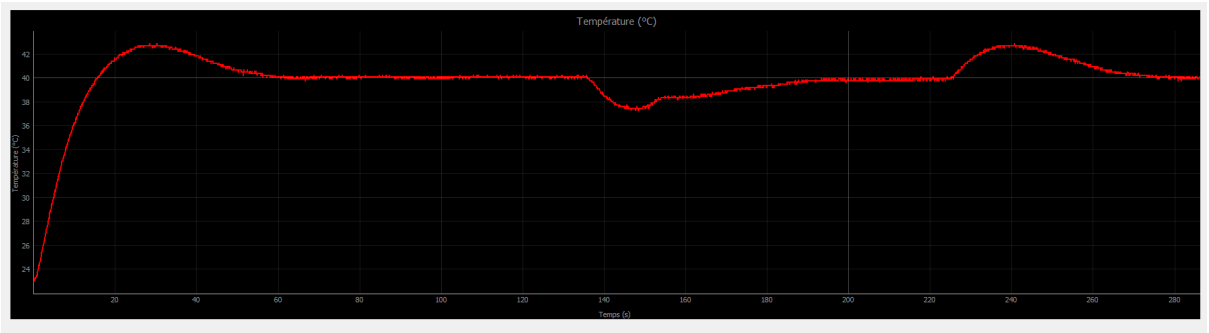


Figure 20 : Graphique de l'essai du premier correcteur

D'après ce graphe, nous pouvons voir qu'il y a un dépassement d'environ 2,5°, l'erreur statique est nulle avant et après perturbation. Lorsque le système est perturbé, il réagit plus rapidement qu'avec le correcteur proportionnel. Nous pouvons calculer le temps de montée à 5%, nous calculons 5% de la différence entre la valeur finale et la valeur initiale soit 0,8°C, nous observons que le système atteint sa valeur finale - 5% à environ 18s. Quand la perturbation cesse, il y a de nouveau un dépassement de même valeur que le premier puis un retour au régime permanent.

Nous réalisons la même étude pour le correcteur suivant, il doit avoir une marge de phase de 30° et w_u=0,05 rad/s, nous obtenons les valeurs et graphiques suivant :

- C0 = 3,5556
- Ti = 7,4927

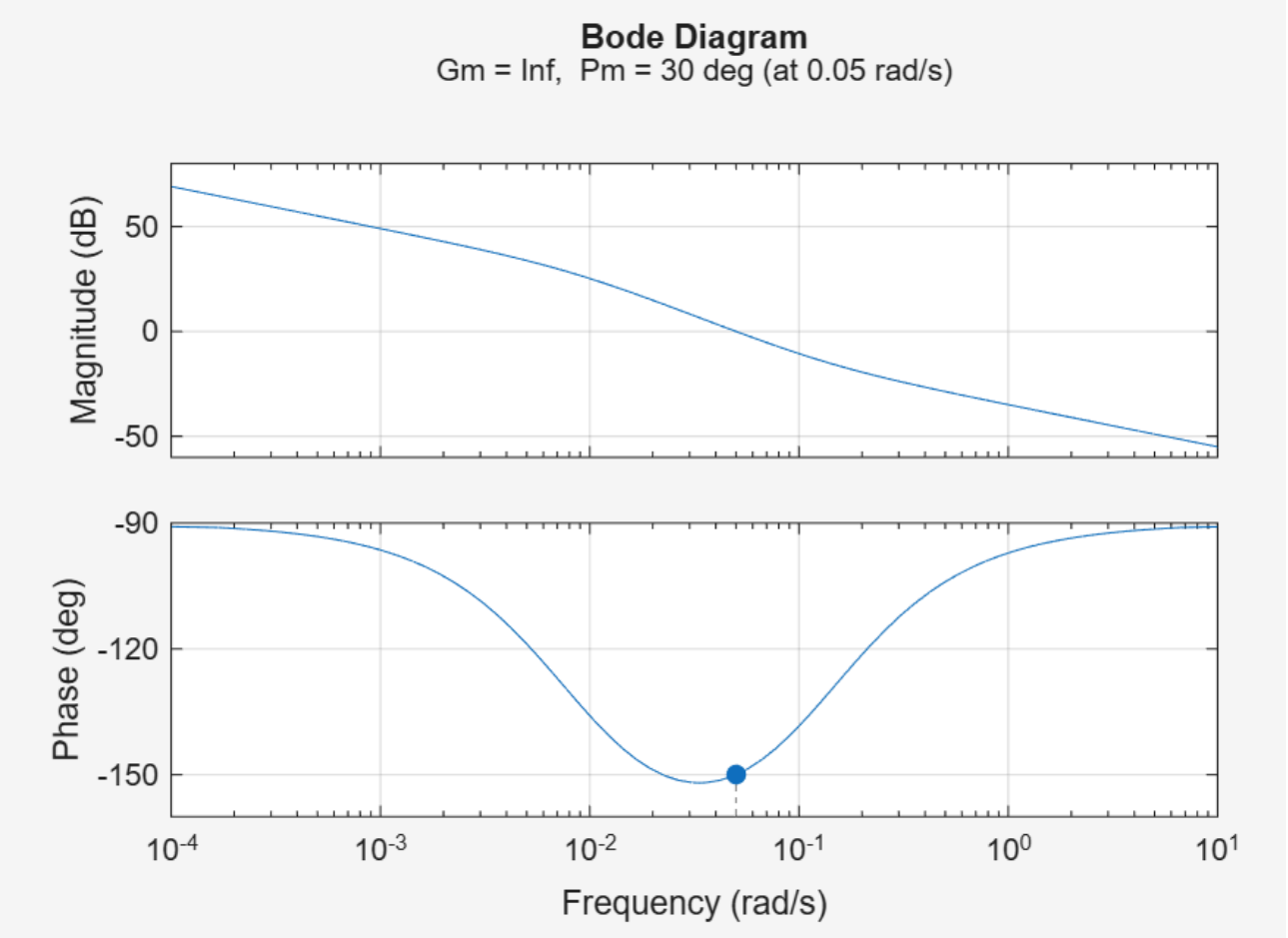


Figure 21 : Simulation Matlab du second correcteur

Nous effectuons un zoom afin de lire les valeurs de pulsation au gain unité ainsi que la marge de phase:

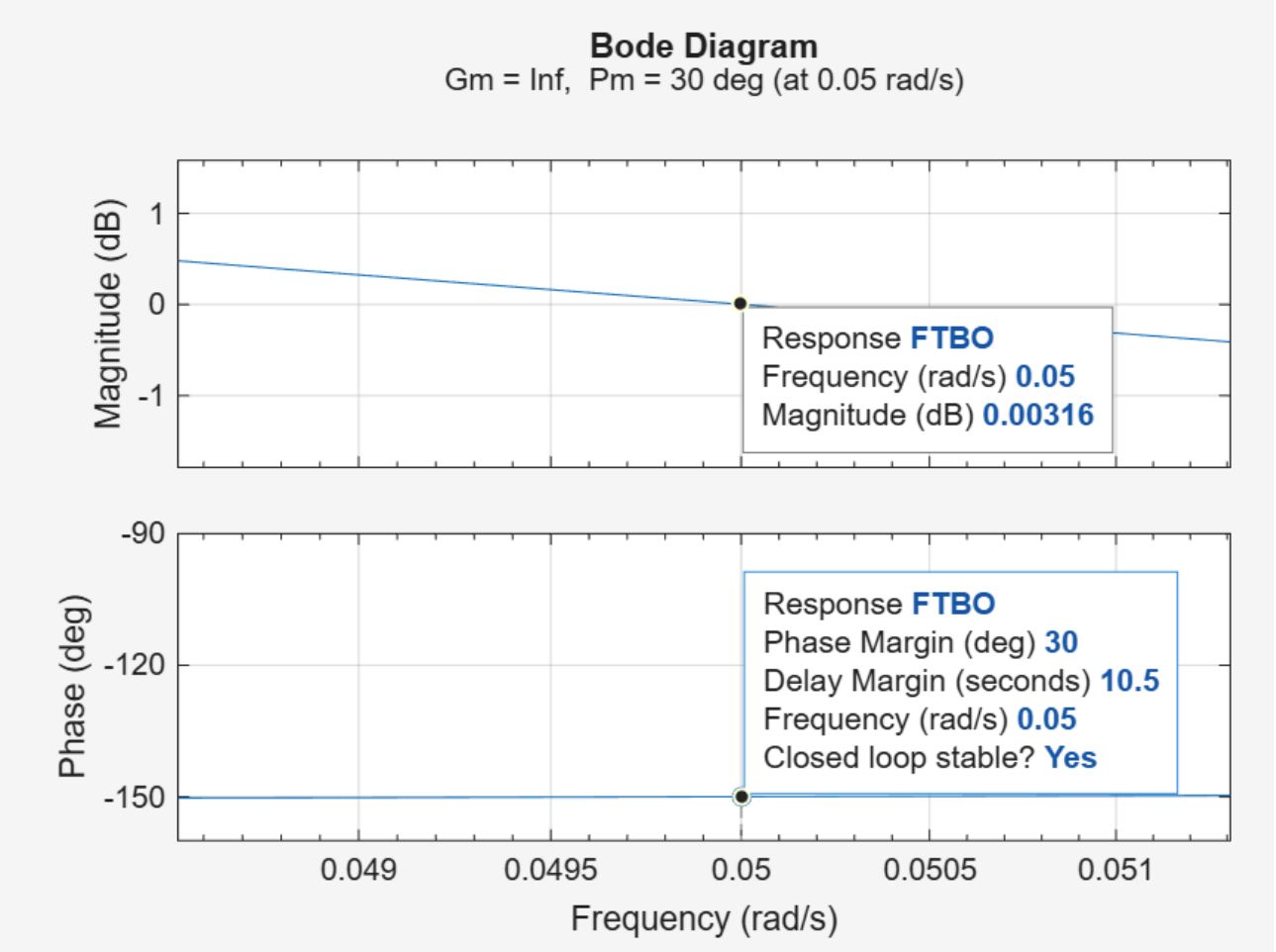


Figure 22 : Simulation Matlab du second correcteur (vue détaillée)

Nous voyons que la pulsation au gain unité est de 0,05 rad/s et la marge de phase est de 30°. Ce correcteur est conforme au cahier des charges.

Nous implémentons de nouveaux ce correcteur dans le microcontrôleur :

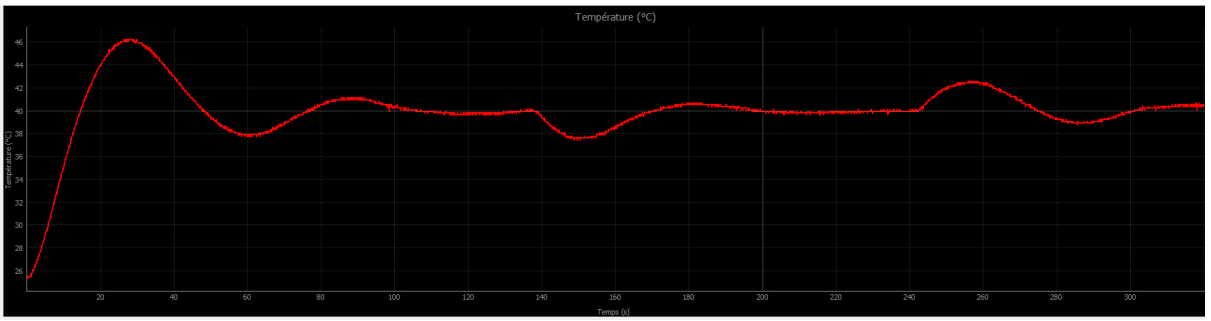


Figure 23 : Graphique de l'essai du second correcteur

D'après la figure 23, nous pouvons voir que le premier dépassement est plus élevé que celui du premier correcteur, il vaut environ 6°C. Son temps de montée à 5% est d'environ 72s. L'erreur statique est nulle avant et après perturbation. Ce système est moins stable mais plus rapide ce qui est cohérent avec les valeurs fixées.

Nous réalisons cette étude une nouvelle fois avec un correcteur dont la marge de phase est 60° et la pulsation au gain unité w_u=0,1 rad/s, nous obtenons les valeurs et graphes suivants :

- C0 = 16,4872
- Ti = 14,4076

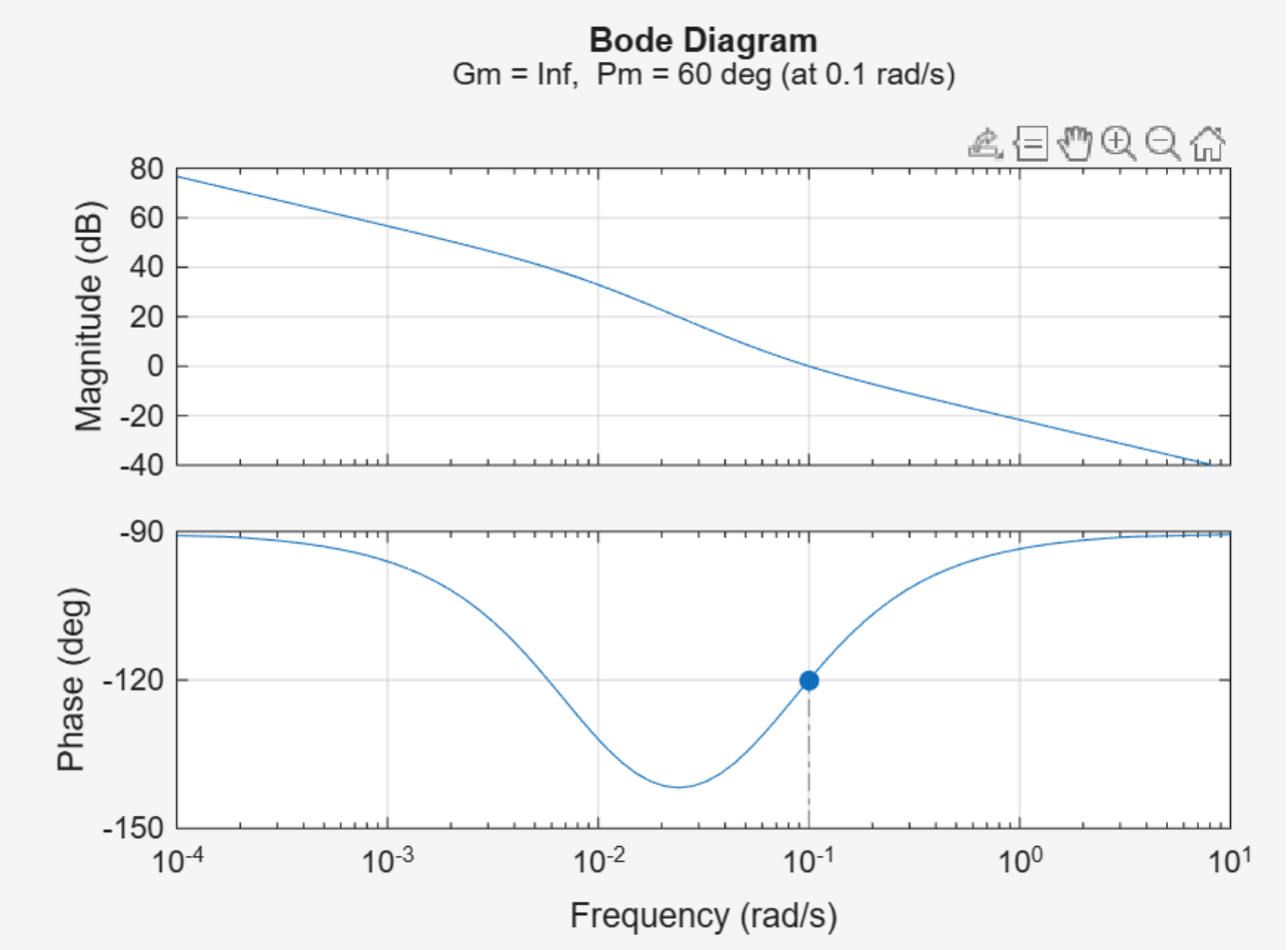


Figure 24 : Simulation Matlab du troisième correcteur PI

Nous effectuons un zoom afin de pouvoir lire la marge de phase ainsi que la pulsation au gain unité :

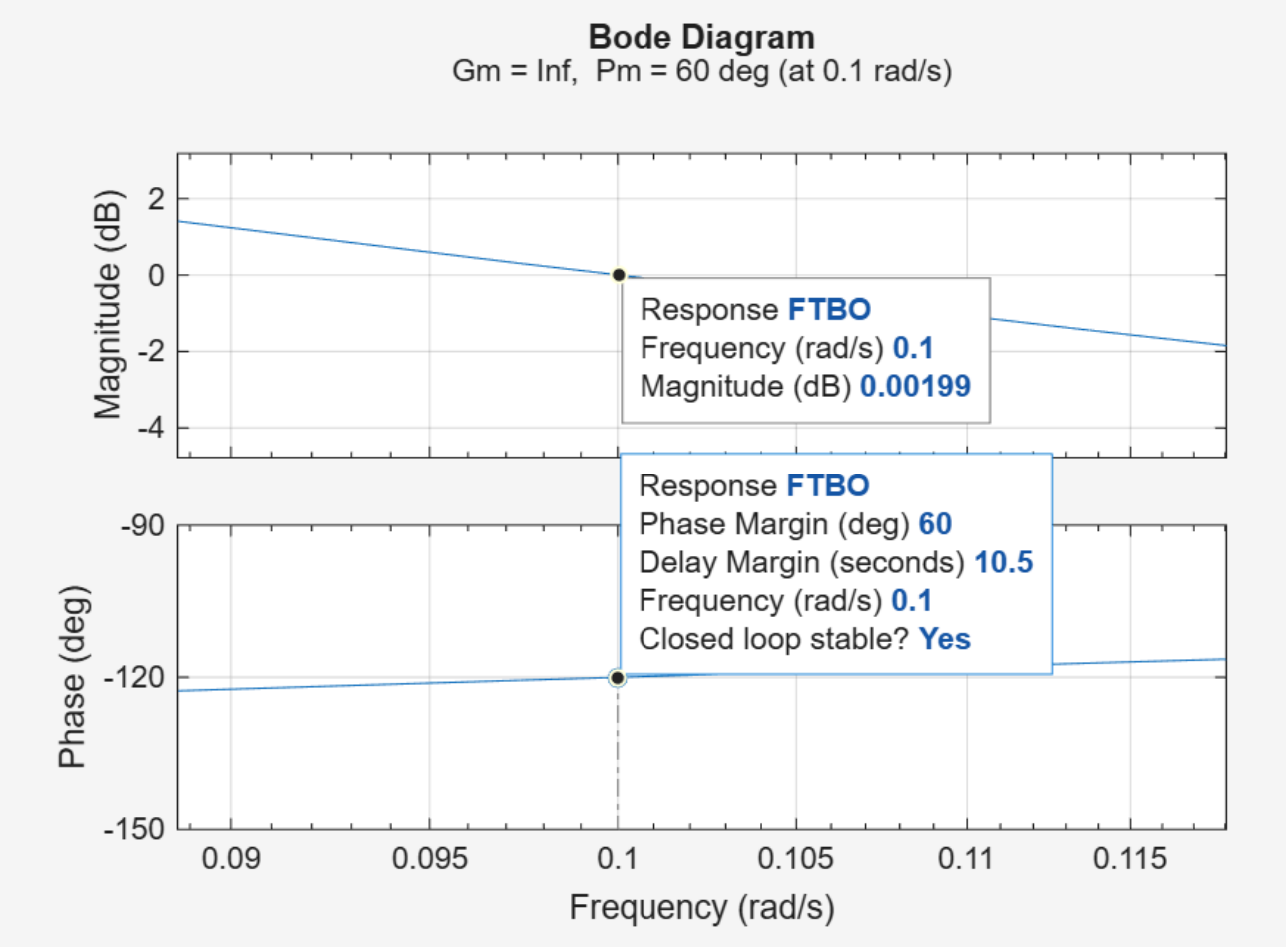


Figure 25 : Simulation Matlab du troisième correcteur(vue détaillée)

Nous voyons que la pulsation au gain unité est de 0,1 rad/s et la marge de phase est de 60°, ce correcteur est donc conforme aux exigences fixées.

Nous réalisons de nouveau cette étude pour le troisième correcteur :

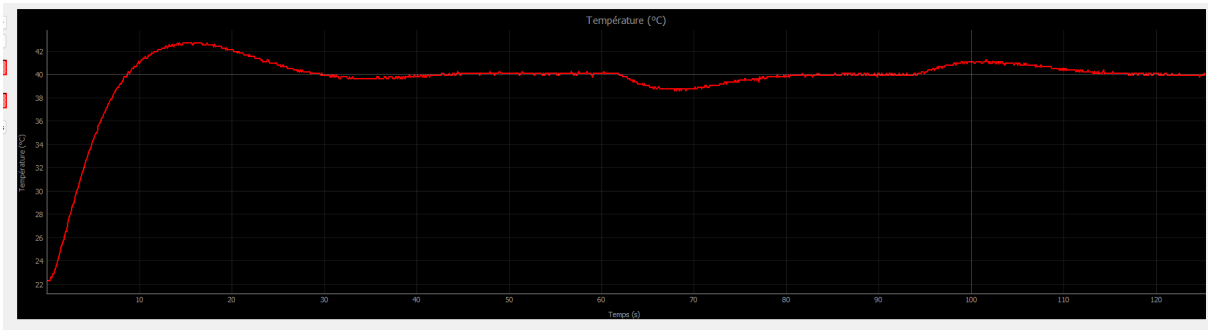


Figure 26 : Graphique de l'essai du troisième correcteur

L'erreur statique est très faible, environ 0,1°C supplémentaire avant perturbation et d'environ 0,1°C en moins après perturbation, probablement due aux imprécisions du microcontrôleur. La valeur du premier dépassement est d'environ 3°C. Le temps de montée à 5% est d'environ 29s. Ce correcteur est moins rapide que le précédent mais plus stable ce qui est cohérent avec les valeurs fixées

Question 1.11 :
On a :

$$C_i(p) = \frac{u(p)}{\epsilon(p)} = C_{0i}\frac{1 + T_i p}{T_i p}$$

On modifie l'équation pour avoir :

$$u(p) \cdot (T_i p) = \epsilon(p) \cdot C_{0i}(1 + T_i p)$$

Puis on développe :

$$T_i \cdot p \cdot u(p) = C_{0i} \cdot \epsilon(p) + C_{0i} \cdot T_i \cdot p \cdot \epsilon(p)$$

En appliquant la transformée inverse de Laplace à l'équation précédente, on obtient :

$$T_i \frac{du(t)}{dt} = C_{0i}\epsilon(t) + C_{0i}T_i\frac{d\epsilon(t)}{dt}$$

Question 1.12
En utilisant l'approximation d'Euler, on peut déduire :

$$\frac{du(t)}{dt} \approx \frac{u(t) - u(t - T_e)}{T_e}$$

$$\frac{d\epsilon(t)}{dt} \approx \frac{\epsilon(t) - \epsilon(t - T_e)}{T_e}$$

On injecte l'approximation de $\frac{du(t)}{dt}$ et $\frac{d\epsilon(t)}{dt}$ dans l'équation de la question 1.1 :

$$T_i \left[\frac{u(t) - u(t - T_e)}{T_e}\right] = C_{0i}\epsilon(t) + C_{0i}T_i \left[\frac{\epsilon(t) - \epsilon(t - T_e)}{T_e}\right]$$

Question 1.15
Le théorème de la valeur finale est : $\lim_{t\to\infty} e(t) = \lim_{p\to 0} p \cdot \epsilon(p)$
L'erreur statique du système est : $\epsilon(p) = \frac{1}{1+C(p)H(p)} \cdot \theta_c(p) - \frac{H_a(p)}{1+C(p)H(p)} \cdot \theta_a(p).$
Pour une consigne en échelon, on a erreur statique : $\epsilon_{stat} = \lim_{p\to 0} p \cdot \left[\frac{1}{1 + C_i(p)H(p)} \cdot \frac{\theta_0}{p}\right] = \lim_{p\to 0} \frac{\theta_0}{1 + C_i(p)H(p)}$
et pour l'erreur de perturbation : $\epsilon_{pert} = \lim_{p\to 0} p \cdot \left[\frac{-H_a(p)}{1 + C_i(p)H(p)} \cdot \frac{A}{p}\right] = \lim_{p\to 0} \frac{-H_a(p) \cdot A}{1 + C_i(p)H(p)}$
Ci(p) tend vers l'infini pour p tend vers 0 donc les limites des deux erreurs tendent vers 1, la sortie recopie l'entrée (la consigne).

Conclusion

Nous remarquons qu'un correcteur PI ne comporte pas d'erreur statique contrairement au correcteur proportionnel. Nous remarquons également que plus la marge de phase est faible plus le correcteur est rapide mais aussi instable. En résumé, ce TP nous a permis de comprendre l'intérêt d'un correcteur PI plutôt qu'un correcteur proportionnel. Nous avons également

développé nos compétences dans l'analyse de graphique et la simulation de correcteur mais également sur l'analyse théorique des correcteurs.